Reproducing and Extending PatchTST: Patch-Based Transformers for Long-Term Time Series Forecasting

Shi Yuan Chu, Ziming Gao, Wenhui Huang, Bing Wen Lim, Yuxiao Wang, Yanting Yang

1 Introduction

Long-term time series forecasting (LTSF) is a critical task in fields ranging from energy grid management to financial market analysis. While Transformer architectures have dominated sequence modeling in natural language processing, their application to LTSF has been hindered by limitations in point-wise tokenization. They treat each individual time step as a token, resulting in $O(L^2)$ attention complexity over the sequence length L. They also have poor inductive bias as a single scalar value provides limited contextual information in isolation.

Early Transformer-based forecasting models attempted to improve efficiency and temporal modeling. Informer introduced ProbSparse attention to reduce the quadratic cost [2], while Autoformer incorporated seasonal-trend decomposition to try and capture long-term temporal patterns [3]. However, neither approach fundamentally departs from point-wise tokenization of time steps. This limitation was empirically highlighted by Zeng et al., who showed that a simple single-layer linear model (DLinear) could match or outperform complex Transformer-based architectures like Autoformer on standard benchmarks [4].

PatchTST addresses this by borrowing an insight from computer vision: just as images are tokenized into patches rather than pixels, time series can be tokenized into contiguous segments of consecutive time steps. Each patch captures a local temporal pattern (a trend, a spike, or a seasonal fragment) rather than a meaningless point, while reducing the number of tokens roughly by a factor of S where S is the stride length. The model further adopts a Channel-Independent (CI) design, where each variable in a multivariate series is processed as a separate univariate sequence through a shared Transformer backbone. This acts as a powerful regularizer and effectively multiplies the available training data by the number of channels. In this project, we reproduce and extend PatchTST to evaluate both its claims and its architectural limitations. Our contributions are:

- **Reproduction from First Principles:** A ground-up PyTorch implementation based on the paper's description, validated on ETTh1 against the paper's reported metrics. We document discrepancies between the paper and the authors' reference code, including the use of RevIN in place of the described instance normalization, and an unconditional padding scheme in the patching formula that we correct.
- **Evaluation on additional datasets:** We evaluate PatchTST, DLinear, and Autoformer on datasets beyond the original benchmarks, including a highly correlated multivariate dataset, to test whether the reported advantages generalize.
- **Cross-Channel Extension:** A cross-channel attention module that relaxes the CI assumption, targeting scenarios where channels share partial temporal patterns and the shared backbone may receive conflicting training signals under strict independence.

2 PatchTST

2.1 Overview

PatchTST is an encoder-only Transformer for multivariate long-term forecasting. Given an input $\mathbf{X} \in \mathbb{R}^{L \times C}$ with lookback length L and C channels, the model predicts $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times C}$ for forecast horizon H. The forward pass consists of seven stages: instance normalization, channel-independent reshaping, patching with embedding, a Transformer encoder, a linear prediction head, reshaping back to multivariate form, and denormalization.

2.2 Instance Normalization

Before entering the model, each sample is independently normalized along the time axis: the per-channel mean and standard deviation are computed, subtracted and divided respectively, and saved for later reversal:

$$\tilde{x}_{t,c} = \frac{x_{t,c} - \mu_c}{\sigma_c}, \qquad \mu_c = \frac{1}{L}\sum_{t=1}^{L} x_{t,c}, \qquad \sigma_c = \sqrt{\frac{1}{L}\sum_{t=1}^{L}(x_{t,c} - \mu_c)^2 + \epsilon} \tag{1}$$

This is distinct from the dataset-level z-score standardization applied during preprocessing, which uses global training set statistics to bring all channels to a common scale. Instance normalization operates per-sample, stripping each individual window's local level and spread so the model focuses on the shape of the signal rather than its absolute position, addressing the non-stationarity inherent in time series data. The statistics are detached from the computation graph—the model cannot learn to manipulate them through backpropagation.

Notably, the official implementation replaces this with RevIN [6], a learned normalization module with trainable affine parameters (γ, β). The paper itself describes only the simpler statistical normalization without learned parameters. We implement the paper's description.

2.3 Channel Independence

The input $\mathbf{X} \in \mathbb{R}^{B \times L \times C}$ is reshaped to $\mathbb{R}^{(B \cdot C) \times L}$ by treating each channel as an independent sample. From this point, the model processes $B \cdot C$ univariate sequences through a shared backbone with no cross-channel interaction. This design rests on the assumption that useful temporal patterns can be learned independently and shared across channels—whether because channels are independent, or because they share sufficiently similar dynamics that a single set of learned temporal patterns serves all channels.

This assumption provides two concrete benefits: regularization (the model cannot overfit to spurious cross-channel correlations) and data efficiency (C times more training sequences from the same data). However, it introduces a failure mode when channels share partial temporal patterns: the shared backbone may receive contradictory gradient signals from channels that look similar in the lookback window but diverge in the prediction horizon. Our extension in Section 5 addresses this limitation.

2.4 Patching

The univariate sequences are segmented into overlapping patches using a sliding window of length P with stride S. The number of patches is:

$$N = \left\lceil \frac{L - P}{S} \right\rceil + 1 \tag{2}$$

When the sequence length does not divide evenly, the final values are replicated to complete the last patch. The original paper defines $N = \lfloor (L-P)/S \rfloor + 2$ and unconditionally pads S

values, which produces an identical result when padding is needed but creates a redundant patch containing fabricated data when the sequence divides cleanly. We adopt the ceiling formulation, which pads only when necessary.

Patching reduces the attention complexity from $O(L^2)$ to $O(N^2)$. With typical settings (L=336, P=16, S=8), this reduces the token count from 336 to 41 (an ≈8× reduction), and the quadratic attention cost by approximately 67×.

2.5 Patch Embedding

Each patch of P raw values is projected to a D-dimensional representation via a shared linear layer. A learnable positional encoding is added to each patch position, providing the model with ordering information that the permutation-invariant attention mechanism would otherwise lack:

$$\mathbf{z}_i = \text{Dropout}(\mathbf{W}_{\text{proj}}\,\mathbf{p}_i + \mathbf{e}_i) \tag{3}$$

where $\mathbf{e}_i$ is the i-th row of a learnable positional embedding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{N\times D}$.

2.6 Transformer Encoder

The encoder consists of M identical layers, each containing two sub-blocks:

1. **Multi-head self-attention**: The input is projected into queries, keys, and values via learned linear transformations, then split across H attention heads each operating in D/H dimensions. Each head computes scaled dot-product attention over the same set of N patch tokens in a different projected subspace, then the heads are concatenated and projected back to D dimensions.

2. **Position-wise feed-forward network**: A two-layer MLP ($D \to d_{ff} \to D$) with GELU activation expands and compresses each token's representation independently.

Both sub-blocks use residual connections followed by BatchNorm—a departure from the LayerNorm used in standard Transformers [1]. The normalization is applied post-residual (post-norm), meaning the residual is added before normalization:

$$\mathbf{z}' = \text{BatchNorm}(\mathbf{z} + \text{Dropout}(\text{MHA}(\mathbf{z}))) \tag{4}$$

$$\mathbf{z}'' = \text{BatchNorm}(\mathbf{z}' + \text{Dropout}(\text{FFN}(\mathbf{z}'))) \tag{5}$$

In a channel-independent setting, the effective batch size is $B \cdot C$ (e.g., $128 \times 7 = 896$), providing the statistical stability that BatchNorm requires.

2.7 Prediction Head and Denormalization

The encoder output $\mathbb{R}^{(B\cdot C)\times N\times D}$ is flattened to $\mathbb{R}^{(B\cdot C)\times(N\cdot D)}$ and projected to the prediction horizon H via a single linear layer with no hidden layers or activation function: $\hat{\mathbf{y}} = \mathbf{W}_{\text{head}}[\mathbf{z}_1; \ldots; \mathbf{z}_N] + \mathbf{b}$. The output is reshaped back to $\mathbb{R}^{B\times H\times C}$ and denormalized using the per-sample statistics saved during instance normalization, restoring the predictions to the original data scale.

3 Reproduction

We implemented PatchTST in PyTorch based on the paper's description, cross-referencing the authors' source code [7] where the paper was ambiguous. The core architecture—patching, multi-head attention, and encoder—was built from scratch; standard components such as the feed-forward dropout placement were adopted from the reference implementation. DLinear and

Autoformer were adopted from Time-Series-Library [8]. To validate correctness, we trained all three models on ETTh1 (7 variables, hourly, 17,420 time steps, standard 12/4/4 month split) using the authors' ETTh1 configuration: L=336, T=96, P=16, S=8, D=16, H=4, d_{ff}=128, M=3 layers, dropout 0.3, Adam (lr=10^{-4}, constant for 3 epochs then 0.9× decay per epoch following the reference code), batch size 128. We deviated from the paper's settings in two respects: seed 42 (vs. 2021) and early stopping patience 10 (vs. 100).

During reproduction, we identified four discrepancies between the paper and its reference code: (1) the patching formula unconditionally pads S values, producing a redundant patch when the sequence divides evenly—we corrected this as described in Section 2.4; (2) the official implementation replaces instance normalization with RevIN, a learned normalization module—we followed the paper; (3) the paper text (Appendix A.1.4) reports dropout 0.2 while the authors' ETTh1 training script uses 0.3—we used 0.3; (4) the reference code applies dropout twice on the attention output—once after the output projection inside the attention module, and again on the residual path in the encoder layer—resulting in an effective drop rate of $1-(1-p)^2 \approx 0.51$ for p=0.3. The paper describes a single dropout per sub-block. We removed the redundant projection dropout to match the paper's description.

Table 1: ETTh1 reproduction validation (pred_len = 96). Lower is better.

Model	MSE (Paper)	MSE (Ours)	MAE (Paper)	MAE (Ours)	Train Time
PatchTST	0.375	0.381	0.399	0.403	196s
DLinear	0.375	0.374	0.399	0.397	9s
Autoformer	0.435	0.528	0.446	0.491	1205s

PatchTST and DLinear are within 2% of the paper's reported MSE (Table 3, PatchTST/42 and DLinear for ETTh1 96 metric, in [5]), confirming faithful implementations. The Autoformer gap is expected: the paper swept six lookback lengths ($L \in \{24, 48, 96, 192, 336, 720\}$) and reported the best result; we only evaluated at L=336.

4 Evaluation on Additional Datasets

To assess whether PatchTST's reported advantages generalize beyond the paper's original benchmarks, we evaluate all three models on two additional standard forecasting datasets (Traffic and Air Quality) and one highly correlated multivariate dataset (Foreign Exchange rates). All experiments use prediction horizon H=96 and the same ETTh1 paper configuration: L=336, P=16, S=8, D=16, H=4, d_{ff}=128, M=3 layers, dropout 0.3.

4.1 Datasets

Our loaders select every numeric column of the source file as an input channel and do not drop any; the channel counts quoted below therefore reflect what the models actually receive, which in a couple of cases includes artefactual columns that we have left in for reproducibility.

Traffic. The Metro Interstate Traffic Volume dataset [9] contains 48,204 hourly records from October 2012 to September 2018, recorded at a sensor on I-94 westbound in Minneapolis–Saint Paul, USA. After preprocessing, 20 numeric columns are passed to the model: the hourly traffic volume itself, four weather covariates (temperature, rainfall, snowfall, cloud cover), calendar integers (day, year), six engineered cyclical time features (hour, day-of-week, month, each as sine/cosine), and seven binary weather-condition flags (holiday, clear, cloudy, rainy, snowy, foggy, squall). The forecast loss is computed over all 20 channels, with `traffic_volume` being the headline channel of interest. Data is split 70% / 15% / 15% for train, validation, and test sets.

Air Quality. The Beijing PM2.5 dataset [10] contains 41,757 hourly records. After preprocessing, 16 numeric columns are passed to the model: the PM2.5 concentration, six meteorological measurements (dew point, temperature, atmospheric pressure, cumulative wind speed, cumulative snow hours, cumulative rain hours), six engineered cyclical time features (hour, month, day-of-week, each as sine/cosine), and three calendar/index columns (the row index `No`, `year`, and `day`) that are retained from the raw file. The wind direction indicators (`wind_NE/NW/SE/cv`) are stored as non-numeric dtypes in our source file and are therefore filtered out by the numeric-column selector. As with Traffic, the forecast loss is computed over all 16 channels, with `pm2.5` being the headline channel of interest. The same 70/15/15 split is applied.

Foreign Exchange (FX). The FX dataset [11] contains 5,978 daily exchange rate records from January 1995 to May 2018. It lists 42 currency columns quoted against the US Dollar, one of which is the `U.S. Dollar` column itself (a constant 1.0 that collapses to zero under our z-score standardization). All 42 columns are passed to the model, so in practice it forecasts 41 foreign-currency-vs-USD rates plus one degenerate constant channel. The forecast loss is computed over all 42 channels, with the `Singapore Dollar` rate being the headline channel of interest. Unlike the two standard benchmark datasets above, the FX dataset is selected specifically because global currency pairs share strong cross-channel correlations driven by common macroeconomic factors, providing a challenging setting for the Channel-Independent PatchTST architecture. The same 70/15/15 split is applied.

4.2 Preprocessing

All three datasets follow the same preprocessing pipeline used for ETTh1. The data is split strictly in chronological order with no shuffling. The validation and test windows are shifted back by L=336 steps to allow a full lookback for the first prediction. Standardization (zero mean, unit variance) is computed on the training portion only and applied to the full dataset, preventing any leakage of future statistics into training. Temporal features (hour-of-day, day-of-week, day-of-month, day-of-year) are extracted from the date column and normalized to $[-0.5, 0.5]$.

4.3 Results

The evaluation results across the additional datasets cleanly highlight the resilience of the Channel-Independent architecture. PatchTST and PatchTST (ACCA) strongly outperformed both DLinear and Autoformer on all three datasets (Traffic, Air Quality, FX). The absolute performance gap is especially stark for Autoformer, which struggled to generalize with the limited small-dataset configurations used.

Caveat on the Autoformer comparison. The ETTh1 paper configuration (D=16, H=4, d_{ff}=128) is severely under-capacity for Autoformer's encoder–decoder, which typically uses D=512 in the original paper. The Autoformer numbers above should therefore be read as a floor rather than a fair baseline; we kept the configuration fixed across all four models to isolate the effect of architecture rather than hyperparameter tuning. The PatchTST-vs-DLinear comparison is unaffected.

4.4 Computational Cost

Beyond forecasting accuracy, the proposal calls for a comparison of the computational cost of training and prediction. Training wall-clock is reported in Tables 1 and 2. For prediction cost we rely on two measurements: the theoretical self-attention complexity of each architecture, and the end-to-end test-set inference time captured by the training pipeline (`test_inference_time`, the wall-clock time taken to run the trained model over the full test loader).

Table 2: Results on additional datasets (pred_len = 96). Lower is better. All runs use the ETTh1 paper config.

Dataset	Model	MSE	MAE	Best Epoch	Train Time
Traffic	PatchTST	0.531	0.461	58	2846.3s
	PatchTST (ACCA)	0.531	0.461	52	2842.0s
	DLinear	0.581	0.517	15	24.6s
	Autoformer	0.685	0.539	10	522.5s
Air Quality	PatchTST	0.222	0.202	50	1828.0s
	PatchTST (ACCA)	0.221	0.201	47	1794.4s
	DLinear	0.278	0.289	11	13.7s
	Autoformer	0.404	0.337	9	428.8s
FX	PatchTST	0.089	0.185	88	1047.0s
	PatchTST (ACCA)	0.089	0.185	89	1044.8s
	DLinear	0.155	0.260	47	8.1s
	Autoformer	0.166	0.287	65	238.0s

The theoretical story follows directly from the architectural choices. Autoformer and Informer both tokenize at the point level, so self-attention scales quadratically with the lookback length: for L=336 they attend over 336 tokens per head. DLinear has no attention at all and scales as $O(L \cdot H)$ per channel. PatchTST patches the sequence with P=16, S=8, yielding N=41 tokens, so attention scales as $O(N^2) = O((L/S)^2)$—for our settings an ≈67× reduction in the attention term relative to point-wise tokenization. The ACCA extension adds a single $C \times C$ linear layer per forward pass, which is negligible next to the encoder ($O(C^2 \cdot N \cdot D)$ compared with $O(M \cdot N^2 \cdot D)$ for the attention stack when $C \ll N$).

In practice, the measured asymmetry in training time between PatchTST and Autoformer (e.g. 2846.3s vs 522.5s on Traffic) is dominated by Autoformer early-stopping after only 10 epochs and PatchTST training for 58; per-epoch wall-clock is comparable because the Autoformer decoder adds work that partially offsets the attention saving. For the prediction-only cost, we surface `test_inference_time` in every run's final summary and in `RESULTS.md`; because the full test sets are small (¡5% of the training wall-clock for every configuration) we do not replicate the numbers here. The ACCA inference overhead is dataset-dependent in our latest runs: it increases PatchTST test inference time on Traffic (2.307s to 2.752s) and Air Quality (1.602s to 1.907s), while being slightly lower on FX (0.549s to 0.542s), so there is no consistent sub-3% overhead pattern across all datasets.

5 Extension: Adaptive Cross-Channel Attention

5.1 Motivation

The Channel-Independent design treats each variable as a fully independent sequence. While this provides strong regularization and data efficiency, it creates a failure mode in datasets where channels share partial—but not identical—temporal dynamics. When two channels exhibit correlated lookback patterns but diverge in their prediction horizons, the shared backbone receives contradictory gradient signals from both channels simultaneously. The model cannot resolve this conflict without access to inter-channel context.

The Foreign Exchange dataset is a controlled setting for this hypothesis: all currency pairs are driven by a common set of global macroeconomic forces, so cross-channel information should in principle be predictive. Evaluating CI PatchTST against a cross-channel-aware variant on FX, while controlling for performance on the standard benchmarks, provides a direct test of

when the independence assumption is a strength versus a limitation.

5.2 Architecture

We introduce the **Adaptive Cross-Channel Attention (ACCA)** module, which is inserted into the existing PatchTST forward pass without modifying the Transformer backbone. After the encoder produces per-channel representations $\mathbf{Z} \in \mathbb{R}^{(B \cdot C) \times N \times D}$, the module reshapes the batch to $\mathbb{R}^{(B \cdot N) \times C \times D}$ to expose the channel axis, computes a cross-channel interaction $\mathbf{H}$, and blends the result back into the original representation via a learned scalar gate α:

$$\mathbf{Z}' = \mathbf{Z} + \alpha \cdot (\mathbf{H} - \mathbf{Z}), \qquad \alpha = \sigma(\alpha_{\text{raw}}) \tag{6}$$

where σ denotes the sigmoid function and α_{raw} is a single learnable scalar parameter. When $\alpha = 0$ the output is identical to vanilla PatchTST; when $\alpha = 1$ the representation is fully replaced by the cross-channel signal. The gate is initialized to $\alpha_{\text{raw}} = -4.6$, giving $\alpha_0 \approx 0.01$, so the model starts as a faithful copy of PatchTST and can learn to open the cross-channel pathway through gradient descent.

The cross-channel interaction $\mathbf{H}$ is computed by a linear map across the channel axis: a shared $C \to C$ linear layer mixes the C channel vectors (patch representations in $\mathbb{R}^D$) at every (B, N) position, directly combining channel representations without explicit attention weights. We deliberately chose this simple linear mixer over a multi-head attention variant: the channel axis is short ($C \leq 42$ in our experiments), and the linear formulation isolates whether any cross-channel signal is beneficial without introducing additional inductive biases from attention heads or positional embeddings.

The module is injected before the prediction head—operating on the patch-level encoder output $\mathbb{R}^{(B \cdot C) \times N \times D}$—so that the mixed representation passes through the existing linear projection to the forecast horizon. This placement allows cross-channel context to influence the final linear projection rather than mixing already-committed forecasts, keeping the head identical to the base PatchTST.

5.3 Results on ETTh1

Table 3 shows the ETTh1 results for PatchTST with and without ACCA. Both ACCA configurations achieve nearly identical MSE and MAE to the base model, indicating that the module does not degrade performance. Critically, the learned gate converges to $\alpha_{\text{effective}} \approx 0.01$ in all runs—very close to its initialization value and near zero—meaning the model learns to keep the cross-channel blending gate almost fully closed. This is a meaningful result in itself: it confirms that for ETTh1, a dataset with only 7 weakly correlated variables, explicit cross-channel mixing provides no useful signal and the CI assumption holds. The true test of the ACCA module is the FX dataset, where the channels are strongly correlated and the gate is expected to open further. However, results on FX show that ACCA yielded nearly identical mean squared error to the base CI model (0.089 for both), suggesting that simple linear cross-channel information does not offer meaningful predictive advantages over the robust channel-independent baseline even here.

Table 3: ETTh1 ACCA ablation (pred_len = 96). Lower is better.

Model	Config	ACCA Type	MSE	MAE	Best Epoch	α_{eff}
PatchTST	ETTh1	—	0.381	0.403	37	—
PatchTST+ACCA	ETTh1	linear	0.381	0.403	37	0.0103
PatchTST+ACCA	default	linear	0.377	0.399	3	0.0102

6 Conclusion

This project reproduced PatchTST from first principles in PyTorch and validated the implementation against the paper's reported results on ETTh1. Our reproduction achieves within 2% of the paper's MSE for PatchTST and DLinear, confirming the correctness of our implementation. During reproduction, we identified and documented four discrepancies between the paper and the authors' reference code: the unconditional padding scheme in the patching formula, the substitution of instance normalization with learned RevIN, a dropout rate discrepancy between the paper text and training scripts, and a doubled dropout on the attention output path. We corrected all four in our implementation to faithfully match the paper's description.

We further proposed and implemented the Adaptive Cross-Channel Attention (ACCA) module, which relaxes the strict channel-independence assumption via a learned alpha-blending gate. On ETTh1, the gate converges to near-zero, confirming that cross-channel mixing is not beneficial for that dataset and that the CI assumption is appropriate there.

Evaluations across the Traffic, Air Quality, and Foreign Exchange datasets reveal a consistent narrative: Channel Independence (CI) is a remarkably resilient structural prior. Even in the FX dataset—comprising 42 strongly correlated currency pairs where cross-channel effects were heavily expected to help—introducing a learnable cross-channel mixing layer (ACCA) failed to provide statistically meaningful forecasting improvements over standard PatchTST. This strengthens the original authors' claim that CI limits harmful covariate shift and representation entanglement better than explicit channel mixing can overcome them for standard forecasting horizons.

References

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

[2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 11106–11115, May 2021, doi: 10.1609/aaai.v35i12.17325.

[3] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, pp. 22419–22430, 2021, doi: 10.48550/arXiv.2106.13008.

[4] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11121–11128, Jun. 2023, doi: 10.1609/aaai.v37i9.26317.

[5] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," in *International Conference on Learning Representations (ICLR)*, 2023, doi: 10.48550/arXiv.2211.14730.

[6] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift," in *International Conference on Learning Representations (ICLR)*, 2022. [Online]. Available: `https://openreview.net/forum?id=cGDAkQo1C0p`. [Accessed: Apr. 20, 2026]

[7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "PatchTST: Official Implementation," GitHub, 2023. [Online]. Available: https://github.com/yuqinie98/PatchTST. [Accessed: Apr. 18, 2026]

[8] H. Wu, T. Hu, Y. Liu, H. Zhou, J. Wang, and M. Long, "TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis," in *International Conference on Learning Representations (ICLR)*, 2023, doi: 10.48550/arXiv.2210.02186. Code: https://github.com/thuml/Time-Series-Library. [Accessed: Apr. 18, 2026]

[9] J. Hogue, "Metro Interstate Traffic Volume," UCI Machine Learning Repository, 2019, doi: 10.24432/C5X60B. [Online]. Available: https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume. [Accessed: Apr. 20, 2026]

[10] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang, and S. X. Chen, "Assessing Beijing's PM2.5 Pollution: Severity, Weather Impact, APEC and Winter Heating," *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 471, no. 2182, p. 20150257, 2015, doi: 10.1098/rspa.2015.0257. Dataset archived at UCI Machine Learning Repository, 2016. [Online]. Available: https://archive.ics.uci.edu/dataset/381/beijing+pm2+5+data. [Accessed: Apr. 20, 2026]

[11] thebasss, "Currency Exchange Rates," Kaggle, 2018. [Online]. Available: https://www.kaggle.com/datasets/thebasss/currency-exchange-rates. [Accessed: Apr. 20, 2026]